

Sistema de Detección de Armas basado en Inteligencia Artificial

Propuesta de Proyecto

Visión por Computadora · Aprendizaje Profundo · Edge Computing
YOLOv8 · Raspberry Pi 5 · NCNN

Autor: Suiza

Año académico 2025-2026

Índice

#	Sección	Pág.
1	Resumen ejecutivo	3
2	El problema que resolvemos	3
3	Objetivos del proyecto	4
4	Solución propuesta	4
5	Tecnologías utilizadas	5
6	Arquitectura del sistema	8
7	Cómo funciona, paso a paso	9
8	El modelo de IA: cómo se entrena	11
9	Estrategia anti-alucinaciones	12
10	Métricas de rendimiento	13
11	Despliegue en Raspberry Pi 5	14
12	Limitaciones honestas	15
13	Próximos pasos	16
14	Preguntas frecuentes (FAQ)	16
15	Glosario	19
16	Conclusión	20

1. Resumen ejecutivo

Este proyecto propone un sistema de detección automática de armas de fuego y armas blancas en tiempo real, utilizando una cámara conectada a una Raspberry Pi 5. El sistema observa de forma continua, detecta si alguien porta un arma, y dispara una alerta inmediata.

La novedad respecto a sistemas similares es doble: **(1)** está entrenado para reconocer armas en distintas posiciones (no solo de lado), y **(2)** incorpora técnicas activas contra falsos positivos ("alucinaciones") para no confundir móviles, mandos, paraguas o bolsas con armas, lo cual es crítico en un entorno con público como un stand o evento.

El hardware objetivo es deliberadamente económico: una Raspberry Pi 5 ejecuta el modelo localmente, sin enviar nada a la nube — preservando privacidad y garantizando funcionamiento aún sin internet.

2. El problema que resolvemos

La detección humana de amenazas armadas en cámaras de seguridad tiene tres limitaciones inherentes:

- **Cobertura limitada:** un operador no puede mirar 20 pantallas a la vez con la misma atención.
- **Fatiga visual:** después de una hora, la atención cae drásticamente.
- **Tiempo de reacción:** entre detectar y avisar pueden pasar segundos valiosos.

La inteligencia artificial puede complementar al operador humano: **nunca se distrae**, mira todas las cámaras a la vez, y reacciona en milisegundos. No reemplaza al humano — le da una ventaja.

El reto técnico, sin embargo, no es trivial. Un sistema mal diseñado genera alertas constantes ante objetos cotidianos (un móvil sostenido en cierta postura, un mando de televisor, un destornillador), pierde credibilidad, y acaba siendo ignorado. Por eso este proyecto pone especial énfasis en la **fiabilidad** y en minimizar las falsas alarmas.

3. Objetivos del proyecto

Objetivo general:

Construir un sistema de visión por computadora que detecte armas en tiempo real con alta fiabilidad, ejecutándose íntegramente en hardware de bajo coste (Raspberry Pi 5).

Objetivos específicos:

- Detectar tres categorías: pistola corta (*handgun*), arma larga (*long_gun*, rifle/escopeta) y arma blanca (*knife*).
- Reconocer armas en diferentes ángulos: de lado, frontal, en perspectiva, vista cenital (CCTV).
- Minimizar falsos positivos sobre objetos confundibles: móviles, mandos, bolsas, paraguas, herramientas de mano.
- Funcionar a una velocidad mínima de 5–10 frames por segundo en Raspberry Pi 5.
- Procesar todo localmente, sin dependencia de servicios externos o conexión a internet.

4. Solución propuesta

El sistema se compone de tres bloques claramente diferenciados:

A. Captura

Una cámara (USB o módulo CSI de la propia Pi5) graba video continuamente. Cada frame es enviado al motor de detección.

B. Inferencia (cerebro del sistema)

El modelo de inteligencia artificial — concretamente **YOLOv8 nano** — analiza cada frame en aproximadamente 100 milisegundos y devuelve una lista de objetos detectados, con su posición (bounding box), clase y nivel de confianza.

C. Respuesta

Si la detección supera el umbral configurado (por ejemplo, 60% de confianza para reducir alarmas erróneas), el sistema activa la alerta: guarda el frame como evidencia, marca el objeto con un recuadro en pantalla, y puede enviar una notificación.

Toda esta cadena ocurre **localmente** en la Pi5. La imagen nunca sale del dispositivo, lo que evita problemas de privacidad y de latencia.

5. Tecnologías utilizadas

El proyecto combina varias herramientas estándares de la industria. A continuación se explica cada una de manera sencilla.

5.1 Python

El lenguaje de programación principal. Python es el estándar en inteligencia artificial porque es legible y tiene un ecosistema enorme de librerías para visión por computadora, aprendizaje profundo y manejo de datos.

Analogía: si pensáramos en construir una casa, Python sería el lenguaje común que hablan todos los obreros — todos lo entienden.

5.2 YOLOv8 (You Only Look Once, versión 8)

YOLO es un modelo de detección de objetos de tiempo real. La sigla significa "solo miras una vez" y describe su filosofía: en lugar de explorar la imagen con muchas ventanas (como hacían algoritmos anteriores), YOLO analiza la imagen **completa una sola vez** y devuelve directamente todas las cajas de los objetos detectados.

Esto lo hace extremadamente rápido y adecuado para video en tiempo real. La versión 8 (YOLOv8), desarrollada por la empresa Ultralytics, es una de las más modernas y precisas.

Versión "nano": usamos la variante más pequeña (3 millones de parámetros, frente a los 11 millones de la versión "small"). Cabe en una Pi5 y es lo bastante rápida para video en directo.

Analogía: imagina a un vigilante que mira a una multitud y, de un vistazo, te dice cuántas personas hay, dónde están, y cuáles llevan gorra. No revisa persona por persona — los ve a todos a la vez. Así trabaja YOLO.

5.3 Ultralytics

Es la librería de Python que implementa y mantiene YOLOv8. Nos permite entrenar el modelo, hacer inferencia y exportarlo a distintos formatos con muy pocas líneas de código.

5.4 PyTorch

Es el motor matemático que está por debajo de YOLO. PyTorch es uno de los frameworks de aprendizaje profundo más utilizados (junto con TensorFlow). Se encarga de las operaciones de cálculo intensivo — multiplicaciones de matrices, redes neuronales — y de aprovechar la GPU cuando hay una disponible.

5.5 OpenCV

Una librería clásica de visión por computadora. La usamos para: leer video de la cámara, redimensionar frames, dibujar las cajas y etiquetas sobre la imagen, y guardar capturas de evidencia.

5.6 ONNX y NCNN (formatos de exportación)

Un modelo entrenado en PyTorch puede ser "traducido" a otros formatos optimizados para dispositivos concretos. En este proyecto usamos dos:

- **ONNX** (Open Neural Network Exchange): un formato estándar y universal. Funciona bien en cualquier plataforma.
- **NCNN**: un motor de inferencia de la empresa Tencent, optimizado específicamente para procesadores ARM como el de la Raspberry Pi 5. En la Pi5 ejecuta el modelo **2-3 veces más rápido** que ONNX.

Analogía: el modelo en PyTorch es como una receta escrita en español. ONNX la traduce a un idioma universal (esperanto), y NCNN la traduce directamente al idioma del cocinero local (la Pi5). Cuanto mejor el traductor, más rápido se cocina.

5.7 Raspberry Pi 5

El ordenador donde se ejecuta el sistema. La Pi5 es la última generación: procesador ARM Cortex-A76 de 4 núcleos a 2.4 GHz, 4 u 8 GB de RAM. Es más potente que un teléfono móvil de gama media y consume unos 7-12 W (frente a los 60-80 W de un PC con GPU).

¿Por qué Pi5 y no un PC?

- Coste: ~80 € frente a 1.000 € de un PC con GPU.
- Consumo: 1-2 € de electricidad al mes funcionando 24/7.
- Tamaño: cabe en una caja pequeña, ideal para instalar discretamente.
- Despliegue: puede colocarse en cualquier sitio con una toma de corriente.

6. Arquitectura del sistema

El flujo de datos es lineal y sencillo:

Eta ­ pa	Com ­ ponen ­ te	Funci ­ ón
1	Cáma ­ ra USB / CSI	Captura el video del entorno en tiempo real.
2	OpenCV (lector de frames)	Convierte el video en frames individuales.
3	Preproce ­ samiento	Redimensiona cada frame a 416×416 píxeles.
4	Modelo YOLOv8n (NCNN)	Analiza el frame y detecta armas. ~100 ms.
5	Filtro de confian ­ za	Descarta detecciones débiles (conf < 0.5).
6	Renderizado	Dibuja la caja y etiqueta sobre la imagen.
7	Alerta	Guarda evidencia y opcionalmente notifica.

Cada frame se procesa de manera independiente. El sistema no necesita memoria del pasado para detectar — eso simplifica el diseño y lo hace robusto: si pierde un frame, el siguiente sigue funcionando.

7. Cómo funciona, paso a paso

Para una persona que va a estudiar el proyecto, este es el flujo detallado de qué pasa cuando un arma aparece frente a la cámara:

Paso 1 — Captura del frame

La cámara, a 30 frames por segundo, envía cada imagen a la Pi5. OpenCV (función `cv2.VideoCapture`) lee el frame y lo deja en memoria como una matriz de píxeles RGB.

Paso 2 — Preprocesamiento

El frame se redimensiona a 416×416 píxeles (el tamaño que espera el modelo nano). Esto es crucial: el modelo siempre recibe el mismo tamaño, independientemente de la resolución original de la cámara. También se normalizan los valores de los píxeles (de 0–255 a 0–1).

Paso 3 — Inferencia (el núcleo de la IA)

El modelo YOLOv8n recibe el tensor preprocesado y, mediante una secuencia de capas convolucionales, extrae características de la imagen (bordes, texturas, formas, patrones de alto nivel como "esto se parece a una empuñadura"). En la última capa, el modelo predice:

- Las coordenadas (x, y, ancho, alto) de cada posible objeto.
- La clase del objeto (knife, handgun, long_gun).
- La confianza, un número entre 0 y 1 que indica cuán seguro está.

Paso 4 — Filtros

El modelo puede devolver muchas predicciones, algunas solapadas o muy débiles. Se aplican dos filtros estándar:

- **Filtro de confianza:** se descartan las predicciones con confianza inferior al umbral (en este proyecto 0.5 o 0.6 para evitar alucinaciones).
- **Non-Maximum Suppression (NMS):** si dos cajas se solapan mucho y predicen el mismo objeto, se conserva la de mayor confianza y se elimina la otra.

Paso 5 — Decisión y respuesta

Si tras los filtros queda al menos una detección, el sistema:

- Dibuja la caja sobre la imagen con OpenCV y la muestra en pantalla.
- Guarda el frame en una carpeta de evidencias con marca temporal.
- Registra el evento en un fichero de log.
- Opcionalmente, envía una notificación (email, Telegram, sonido).

Todo este ciclo (paso 1 al 5) ocurre en aproximadamente **150-200 milisegundos por frame** en la Pi5, lo que da una tasa de unos 5-7 FPS reales — suficiente para vigilancia.

8. El modelo de IA: cómo se entrena

Un modelo de IA no "sabe" detectar armas por defecto. Se le enseña con ejemplos. Este proceso se llama **entrenamiento supervisado**.

8.1 El dataset

Un dataset es un conjunto de imágenes con etiquetas. Cada etiqueta indica: "en esta imagen, en estas coordenadas, hay una pistola". El modelo aprende viendo miles de estos ejemplos.

Este proyecto utiliza un dataset combinado de varias fuentes públicas, con un total de aproximadamente **35.000 imágenes** en el conjunto de entrenamiento.

8.2 División del dataset

Las imágenes se reparten en tres grupos, como se hace en cualquier experimento científico:

- **Entrenamiento (~85%)**: el modelo las ve y aprende de ellas.
- **Validación (~10%)**: el modelo nunca las ve durante el entrenamiento. Se usan para medir cómo de bien generaliza.
- **Test (~5%)**: reservadas para la evaluación final.

8.3 Augmentación de datos

Para que el modelo no "memorice" las imágenes sino que aprenda los conceptos, se aplican transformaciones aleatorias en cada época:

- **Espejo horizontal**: una pistola en mano derecha se vuelve mano izquierda.
- **Rotación**: hasta 30 grados, para que aprenda armas en cualquier ángulo.
- **Perspectiva**: simula vistas frontales y en escorzo.
- **Mosaic**: combina cuatro imágenes en una sola para enseñar al modelo a manejar múltiples objetos en distintas escalas.

8.4 El proceso de entrenamiento

El modelo se entrena por **épocas** (cada época = una pasada completa sobre todo el dataset). En cada época:

- El modelo predice sobre cada imagen.
- Se compara la predicción con la etiqueta real ("ground truth").
- Se calcula el error (loss).
- El optimizador ajusta los millones de pesos internos del modelo para reducir ese error.

Tras 30 épocas, el modelo ha visto cada imagen 30 veces y ha refinado sus pesos hasta minimizar el error. El entrenamiento de la versión small tomó aproximadamente **2,5 horas en una GPU RTX 4060**.

9. Estrategia anti-alucinaciones

Un modelo que solo ha visto pistolas en su entrenamiento aprenderá a detectar pistolas — pero también puede creer ver pistolas donde no las hay. Por ejemplo: un móvil sostenido vertical, un mando de televisión, una bolsa oscura, un destornillador.

A estos errores se les llama **falsos positivos** o, coloquialmente, **alucinaciones** del modelo. Son especialmente peligrosos en un stand con público.

9.1 Hard negatives

La técnica fundamental contra las alucinaciones es entrenar al modelo explícitamente con imágenes **parecidas a un arma pero que no lo son**. A estas imágenes se las llama *hard negatives* (negativos difíciles).

En este proyecto hemos incluido más de **2.500 imágenes de hard negatives** en el entrenamiento:

Fuente	Imágenes	Tipo de distractor
Open Images Dataset	846	Personas, móviles, mandos, herramientas
Phone Detection (HF)	605	Móviles en distintas poses
Rifle-vs-Umbrella	120	Paraguas (silueta similar a rifle en CCTV)
Handgun-vs-BagOfChips	110	Bolsa de chips oscura confundible
TOTAL	≈ 1.681 hard neg	(+846 ya estaba en el original)

9.2 Penalización extra a falsos positivos

En el entrenamiento aumentamos el peso del componente de *classification loss* (de 0.5 a 1.0). Esto hace que cada vez que el modelo se equivoque y diga "hay arma" cuando no la hay, se le penalice el doble — incentivándolo a ser más conservador.

9.3 Umbral de confianza en inferencia

En despliegue se sube el umbral de confianza de 0.25 (el valor por defecto) a 0.5 o 0.6. Esto significa que solo se reporta una detección si el modelo está al menos un 50-60% seguro. Reduce drásticamente las falsas alarmas, a costa de algunos verdaderos positivos de baja confianza (casos extraños).

9.4 Resultado

La combinación de hard negatives + penalización extra + umbral alto ha aumentado la precisión del modelo nano en **+2,4 puntos porcentuales**, lo que en un escenario con público se traduce en muchas menos falsas alarmas.

10. Métricas de rendimiento

Para evaluar el rendimiento de un modelo de detección se utilizan varias métricas estándar. Explicación breve de cada una:

Métrica	Qué significa
Precision	De todas las veces que el modelo dijo "arma", ¿en qué porcentaje acertó?
Recall	De todas las armas reales presentes, ¿qué porcentaje detectó?
mAP@50	Precisión media considerando una caja como acierto si solapa $\geq 50\%$ con la real.
mAP@50-95	Promedio de la precisión con umbrales de solape entre 50% y 95%. Métrica más estricta.

10.1 Resultados obtenidos

Tras el entrenamiento sobre el dataset extendido, los modelos finales (versión 4) obtuvieron:

Modelo	Params	mAP@50	Precision	Recall	Uso recomendado
Nano v4	3 M	0,852	0,875	0,758	Raspberry Pi 5
Small v4	11 M	0,879	0,879	0,792	PC con GPU

Lectura: el modelo nano (que va a la Pi5) tiene una precisión del **87,5%**: de cada 100 veces que dice "arma", 87,5 son aciertos. Eso es excelente para un sistema desplegado en hardware barato.

El recall del 75,8% significa que detecta 3 de cada 4 armas que aparecen. El 25% restante son casos límite (oclusión parcial, distancia muy lejana, iluminación pésima) — el sistema no es infalible, pero esa cifra es competitiva con sistemas comerciales.

11. Despliegue en Raspberry Pi 5

El despliegue final sigue estos pasos:

- 1. Se exporta el modelo entrenado al formato NCNN.
- 2. Se copian los archivos (~12 MB) a la Pi5 vía SCP.
- 3. Se instala Ultralytics y NCNN en la Pi5 (`pip install ultralytics ncnn`).
- 4. Se conecta una cámara USB o el módulo CSI de la Pi.
- 5. Se lanza el script de inferencia.

Ejemplo de comando de ejecución en la Pi5:

```
yolo predict model=best_ncnn_model imgsz=416 source=0 conf=0.5
```

El parámetro `source=0` indica que use la cámara conectada. `conf=0.5` es el umbral de confianza para reducir falsos positivos.

12. Limitaciones honestas

Ningún sistema de IA es perfecto. Es importante reconocer sus limitaciones para no generar expectativas irreales:

- **Armas pequeñas a gran distancia:** si la pistola ocupa menos de 20×20 píxeles en la imagen, la detección es poco fiable.
- **Oclusiones parciales:** un arma escondida bajo la chaqueta o en el bolsillo será invisible para el sistema.
- **Iluminación extrema:** escenas muy oscuras o contraluces fuertes degradan el rendimiento.
- **Armas no convencionales:** el modelo no detectará armas fabricadas en casa, armas blancas atípicas o disfraces.
- **Latencia:** en la Pi5 la inferencia tarda ~100 ms; si el arma aparece y desaparece en menos tiempo, puede no detectarse.
- **El sistema NO sustituye al operador humano.** Es una capa de apoyo, no un reemplazo.

13. Próximos pasos

El proyecto se concibe como iterativo. Líneas de mejora previstas:

- Capturar 50–100 frames del escenario real (stand) y reentrenar el modelo con esos datos para adaptarlo al contexto específico.
- Añadir tracking entre frames (no solo detección frame a frame) para reducir alertas momentáneas.
- Integrar con un sistema de notificación: Telegram, email, SMS o panel web.
- Probar versiones cuantizadas a INT8 para acelerar aún más en la Pi5.
- Añadir reconocimiento de comportamiento (no solo objeto): apuntar, blandir, ocultar.

14. Preguntas frecuentes (FAQ) — para estudiar

Estas son las preguntas que con mayor probabilidad puede hacer un tribunal o jurado durante la exposición, con respuestas modelo.

P1. ¿Por qué YOLO y no otro modelo de detección?

YOLO es el estado del arte en detección de objetos en tiempo real. Otros modelos (Faster R-CNN, SSD) son o más lentos o menos precisos. YOLOv8 ofrece el mejor equilibrio entre velocidad y precisión, especialmente en hardware modesto como la Pi5.

P2. ¿Por qué la versión "nano" y no la "small" o "large"?

La nano tiene 3 millones de parámetros y cabe holgadamente en la Pi5. La small tiene 11 millones y la Pi5 la ejecutaría a 1-2 FPS, insuficiente para video. La nano alcanza 5-7 FPS reales, suficiente para vigilancia.

P3. ¿Qué pasa si el modelo se equivoca y dice que hay arma cuando no la hay?

Para eso usamos varias técnicas: (1) entrenamos el modelo con *hard negatives* — imágenes de objetos parecidos pero que no son armas; (2) penalizamos extra los falsos positivos durante el entrenamiento; (3) en despliegue usamos un umbral de confianza alto (0.5 o 0.6). El resultado es una precisión del 87,5%.

P4. ¿Y si el modelo no detecta un arma real?

Eso es un *falso negativo*. El recall del 75,8% significa que detectamos 3 de cada 4 armas reales. El 25% restante son casos extremos (oclusión, distancia, iluminación pésima). Por eso el sistema se concibe como apoyo, no como reemplazo, del operador.

P5. ¿Por qué Raspberry Pi 5 y no un servidor potente?

Tres razones: (1) coste — 80 € vs 1.000 €; (2) consumo — ~10 W vs ~80 W; (3) privacidad — todo se procesa localmente, las imágenes no salen del dispositivo. Para un stand o instalación pequeña es ideal.

P6. ¿Cómo se entrena un modelo de IA?

Se le muestran miles de imágenes etiquetadas (con cajas marcando dónde está el arma). El modelo va ajustando millones de "pesos" internos para minimizar el error entre lo que predice y la etiqueta real. Tras suficientes iteraciones (épocas), el modelo aprende a generalizar y puede detectar armas en imágenes nuevas que nunca vio.

P7. ¿Cuánto tiempo y recursos cuesta entrenarlo?

El modelo nano se entrena en aproximadamente 1,5 horas en una GPU RTX 4060. El small en 2,5 horas. Una vez entrenado, ya no hace falta más GPU: la inferencia se hace en la Pi5 con su CPU.

P8. ¿Qué es ONNX y por qué exportamos a NCNN?

Son formatos de modelo optimizados. El modelo se entrena en PyTorch, pero PyTorch en la Pi5 es lento. NCNN está específicamente optimizado para procesadores ARM (los de la Pi y los teléfonos) y ejecuta el modelo 2-3 veces más rápido que PyTorch puro.

P9. ¿De dónde sacáis las imágenes para entrenar?

De datasets públicos: Open Images (Google), Roboflow Universe, HuggingFace. Combinamos varios — más de 35.000 imágenes en total — y mapeamos sus etiquetas a nuestras tres clases (knife, handgun, long_gun). Todas las licencias son compatibles con uso académico.

P10. ¿Es legal grabar a la gente con este sistema?

El sistema procesa imágenes en tiempo real sin guardar el video completo — solo guarda frames concretos cuando detecta una alerta, y eso ya cumple la finalidad legítima de seguridad. En un despliegue real habría que cumplir el RGPD: cartelera visible, política de privacidad, plazo de conservación limitado. En esta propuesta es un stand académico y aplica el consentimiento implícito del visitante.

P11. ¿Qué pasa si alguien intenta engañar al sistema?

Hay ataques conocidos contra modelos de visión (parches adversariales, oclusiones intencionadas). El modelo no es robusto ante un atacante sofisticado. Pero el escenario de uso — alerta temprana en eventos — no asume un adversario que conozca el sistema.

P12. ¿Por qué tres clases (knife, handgun, long_gun) y no más?

Cuanto más clases, más datos hace falta y más se confunde el modelo. Tres categorías cubren el 99% del caso de uso (cuchillo, pistola corta, arma larga). Subdividir entre pistola/revólver o entre rifle/escopeta no aporta valor para una alerta.

P13. ¿Funciona en oscuridad / de noche?

Mal. El modelo está entrenado con imágenes en condiciones de luz normal. Para uso nocturno habría que: (a) usar cámara con visión infrarroja, y (b) reentrenar con imágenes IR. Es viable como iteración futura.

P14. ¿Cómo medís la fiabilidad del sistema?

Reservamos un 10% del dataset (~865 imágenes) que el modelo no ve durante el entrenamiento — son el **set de validación**. Sobre ese conjunto medimos precisión, recall y mAP. Son métricas estándar en la comunidad de visión por computadora.

P15. ¿Qué tecnologías has usado tú directamente, sin ayuda de IA?

La arquitectura del pipeline (captura → preprocesamiento → inferencia → respuesta), la selección de datasets, la decisión de usar hard negatives, los hiperparámetros del entrenamiento (learning rate, batch size, augmentation), y la integración con la Pi5. La IA generativa ayudó a escribir partes del código boilerplate, pero las decisiones técnicas son propias.

P16. ¿Y si el tribunal pregunta algo que no sé?

Honestidad. "No tengo ese dato concreto memorizado, pero lo podría calcular / comprobar en X minutos" es una respuesta válida. Reconocer un límite es preferible a inventar.

15. Glosario

Términos técnicos que conviene tener claros:

Término	Definición
Bounding box	Caja rectangular alrededor del objeto detectado, definida por (x, y, w, h).
Confianza	Número entre 0 y 1 que indica cuán seguro está el modelo de la detección.
Dataset	Conjunto de imágenes con sus etiquetas, usado para entrenar el modelo.
Edge computing	Ejecutar IA en dispositivos cercanos al usuario (Pi5) en lugar de la nube.
Época	Una pasada completa del modelo sobre todo el dataset durante el entrenamiento.
FPS	Frames Per Second. Velocidad a la que el sistema procesa video.
Hard negative	Imagen sin armas pero visualmente parecida, usada para evitar falsos positivos.
Inferencia	El acto de usar un modelo ya entrenado para hacer predicciones.
Loss	Función matemática que mide el error del modelo. Se minimiza al entrenar.
mAP	Mean Average Precision. Métrica estándar para evaluar modelos de detección.
NCNN	Motor de inferencia optimizado para procesadores ARM (Pi5, móviles).
NMS	Non-Maximum Suppression. Elimina cajas duplicadas que solapan.
ONNX	Formato estándar e interoperable para modelos de IA.
Parámetro / peso	Número interno del modelo que se ajusta durante el entrenamiento.
Recall	De todas las armas reales, qué porcentaje detectó el modelo.
Tensor	Una matriz multidimensional de números (la representación interna de una imagen).
YOLO	You Only Look Once. Familia de modelos de detección en una sola pasada.

16. Conclusión

Esta propuesta presenta un sistema de detección de armas con IA diseñado bajo tres principios: **fiabilidad** (minimizando falsas alarmas), **accesibilidad** (corre en hardware de 80 €) y **privacidad** (todo el procesamiento es local).

Las decisiones técnicas — elegir YOLOv8 nano, usar NCNN para la Pi5, ampliar el dataset con hard negatives, ajustar el umbral de confianza — están todas orientadas a un objetivo concreto: un sistema que sea útil en un escenario real con público, no solo en un benchmark de laboratorio.

El resultado es un modelo nano con **87,5% de precisión** y **75,8% de recall**, ejecutándose a 5-7 FPS en Raspberry Pi 5. Cifras competitivas con sistemas comerciales, obtenidas con herramientas open source y datasets públicos.

Como toda propuesta, el siguiente paso lógico es la prueba en campo: instalar el sistema en el stand, recoger datos reales, e iterar.

Fin del documento.